

Wprowadzenie do Sieci Przemysłowych

Laboratorium: Protokół Modbus TCP w Pythonie

Czas trwania: 2 × 45 min | **Wymagania:** Python 3.x, pakiet pymodbus | **Autor:** inż. Anna Wójcik

Cel laboratorium: Zapoznanie z zasadą transmisji danych w systemach automatyki przemysłowej (OT) na przykładzie protokołu Modbus TCP. Ćwiczenie obejmuje inspekcję rejestrów pamięci wirtualnego sterownika PLC, cykliczne odpytywanie aparatury obiektowej oraz implementację dyskretnego regulatora dwupołożeniowego z histerezą w języku Python.

1. Wstęp

1.1 Architektura systemów sterowania: od warstwy polowej do kodu

W klasycznej informatyce większość systemów opiera się na architekturze chmurowej, mikrouślugach i transmisji danych w formatach tekstowych (np. REST API z formatem JSON). W automatyce przemysłowej priorytety są inne: liczy się bezpośrednia kontrola nad fizycznym procesem, wysoka niezawodność oraz praca w pętli deterministycznej.

Z czym faktycznie komunikujemy się w przemyśle? Centralnym punktem większości instalacji procesowych jest **sterownik PLC** (*Programmable Logic Controller*). To odporny na zakłócenia elektromagnetyczne i trudne warunki środowiskowe komputer przemysłowy, który wykonuje swój program w cyklu (ang. *PLC scan cycle*):

1. **Odczyt wejść:** próbkowanie sygnałów z aparatury pomiarowej wpiętej do modułów I/O (np. napięcie 0–10 V z przetwornika poziomu, stany binarne 24 V z krańcówek),
2. **Wykonanie logiki sterowania:** ewaluacja programu napisanego najczęściej w językach normy IEC 61131-3 (najczęściej tekstowy *Structured Text* lub graficzny schemat drabinkowy *Ladder Diagram*),
3. **Uaktualnienie wyjść:** wysłanie sygnałów sterujących do elementów wykonawczych (załączenie cewki elektrozaworu, wysłanie zadanej prędkości obrotowej do falownika pompy).

Poza sterownikami PLC w magistralach przemysłowych węzłami sieciowymi bywają bezpośrednio:

- **przeмиenniki częstotliwości (falowniki):** sterujące pracą silników asynchronicznych i serwonapędów,
- **moduły rozproszonych wejść/wyjść (Remote I/O):** zbierające sygnały lokalnie przy maszynie, by ograniczyć okablowanie,
- **inteligentna aparatura pomiarowa (smart transmitters):** udostępniające w rejestrach nie tylko pomiar główny, lecz także diagnostykę obiektu.

Rola systemów nadrzędnych (SCADA / DCS) i miejsce Pythona: sterowniki PLC rzadko pracują w całkowitej izolacji. Nadzorują je systemy nadrzędne klasy **SCADA** (*Supervisory Control and Data Acquisition*), serwery baz danych procesowych (*Historian*) oraz panele operatorskie **HMI** (*Human-Machine Interface*). To one odpytują sterowniki protokołami takimi jak Modbus TCP, OPC UA czy Profinet, aby zbierać telemetrię, archiwizować trendy, zarządzać alarmami i przysyłać nastawy technologiczne zadane przez operatora.

Współcześnie Python stał się standardem inżynierskim do zadań brzegowych (*Edge Computing*), analizy predykcyjnej (wykrywanie zużycia pomp na podstawie drgań i prądów), a przede wszystkim

do **emulacji i testów obiektowych**. Zamiast budować fizyczny zbiornik cieczy lub ryzykować uszkodzenie zaworów w hali fabrycznej, piszemy wirtualny węzeł Modbus emulujący dynamikę procesu.

W tym laboratorium Twój komputer pełni jednocześnie dwie role:

- **serwer Modbus** (`server_tank.py`): emuluje sterownik PLC spięty z fizycznym obiektem (zbiornikiem). Udostępnia w rejestrach pomiary poziomu, stan czujnika alarmowego oraz przyjmuje rozkazyysterowania zaworu.
- **klient Modbus** (`controller.py`): program sterujący w Pythonie, który pełni rolę regulatora czasu rzeczywistego i prostego panelu monitoringu.

1.2 Po co nam sieci przemysłowe?

Cecha / Kryterium	Świat IT (Biurowy / Web)	Świat OT (Automatyka / Przemysł)
Główny priorytet	Poufność i przepustowość danych	Dostępność procesu i bezpieczeństwo ludzi (<i>Safety</i>)
Reżim czasowy	<i>Best-effort</i> (akceptowalne zmienne opóźnienia)	Determinizm (gwarantowany czas odpowiedzi T_s , niski jitter)
Typowa transmisja	Rzadkie pakiety o dużym rozmiarze (pliki, JSON)	Częste, mikroskopijne ramki cykliczne (stany I/O, bity, słowa 16-bit)
Cykl życia węzła	Wymiana sprzętu co 3–5 lat	Praca ciągła 24/7 przez 15–30 lat bez restartu
Skutek awarii sieci	Błąd HTTP 500, ponowienie żądania	Zniszczenie maszyny, uszkodzenie instalacji, skażenie środowiska

1.3 Model Danych Modbus (Przestrzenie Adresowe)

Przestrzeń (Data Model)	Szerokość	Dostęp	Przykładowe zastosowanie
Discrete Inputs	1 bit	Read-Only	Czujniki krańcowe, stany alarmowe, czujniki zbliżeniowe
Coils (Cewki)	1 bit	Read/Write	Wysterowanie elektrozaworów, styczników, diod LED
Input Registers	16 bit	Read-Only	Pomiary analogowe (czujnik temperatury, przetwornik ciśnienia)
Holding Registers	16 bit	Read/Write	Nastawy algorytmów (setpoints), progi histerezy

1.4 Podstawowe Kody Funkcji (Function Codes)

Ramka protokołu Modbus w polu **Function Code** (PDU) definiuje rodzaj żądanej operacji. W bibliotece `pymodbus` metody klienta są bezpośrednim odzwierciedleniem tych kodów:

Kod (HEX)	Kod (DEC)	Nazwa operacji	Odpowiednik w pymodbus
0x01	01	Read Coils	<code>client.read_coils(address, count)</code>
0x02	02	Read Discrete Inputs	<code>client.read_discrete_inputs(address, count)</code>
0x03	03	Read Holding Registers	<code>client.read_holding_registers(address, count)</code>
0x04	04	Read Input Registers	<code>client.read_input_registers(address, count)</code>
0x05	05	Write Single Coil	<code>client.write_coil(address, value)</code>
0x06	06	Write Single Register	<code>client.write_register(address, value)</code>
0x0F	15	Write Multiple Coils	<code>client.write_coils(address, values)</code>
0x10	16	Write Multiple Registers	<code>client.write_registers(address, values)</code>

1.5 Anatomia Ramki Modbus TCP (MBAP + PDU)

W przeciwieństwie do protokołów webowych (np. HTTP) narzut nagłówka w Modbus TCP jest minimalny. Cała ramka zapytania o odczyt rejestru to zaledwie 12 bajtów przesyłanych w payloadzie TCP:

MBAP Header (7 bajtów)				PDU (5 bajtów)	
Transaction ID (2B)	Protocol ID (2B, zawsze 0)	Length (2B)	Unit ID (1B)	Function Code (1B)	Data (Rejestr + Ilość)

Przykładowa ramka heksadecymalna (Odczyt 1 rejestru od adresu 0):

[00 01] [00 00] [00 06] [01] [04] [00 00] [00 01]

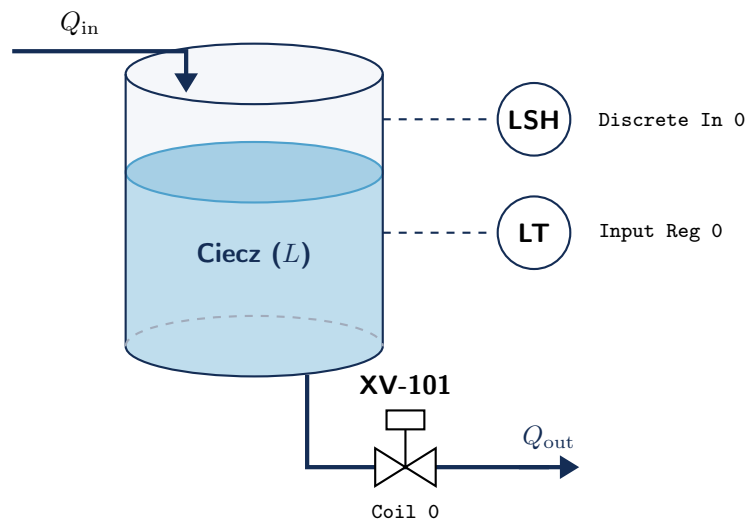
- 00 01 – Unikalny identyfikator transakcji (odpowieź serwera zawiera ten sam ID).
- 04 – Kod operacji (*Read Input Registers*).
- 00 00 oraz 00 01 – Adres początkowy (0) oraz liczba żądanych rejestrów (1).

Pułapka Inżynierska: konwencja adresacji (0-based vs 1-based)

W dokumentacji automatyki i sterowników PLC rejestry trzymane często oznacza się schematem Modicon: np. **40001** lub **400001**. Cyfra **4** oznacza typ rejestru (*Holding Register*), a liczba **1** to pierwszy element (indeksowanie od 1). Protokół na poziomie binarnym (oraz w pymodbus) indeksuje od zera, zatem logiczny rejestr **40001** ma fizyczny adres ramki **0** (`address=0`).

1.6 Opis procesu technologicznego i schemat

Obiektem sterowania jest przemysłowy bufor cieczy z ciągłym, grawitacyjnym dopływem medium ($Q_{in} = \text{const}$). Zadaniem układu automatyki jest utrzymywanie objętości cieczy w strefie bezpiecznej oraz zabezpieczenie przed zalaniem hali.



Oznaczenia aparatury (standard ISA-5.1):

- **LT (Level Transmitter)**: ciągły przetwornik poziomu cieczy → Input Register 0 (0–1000 l).
- **LSH (Level Switch High)**: dyskretny czujnik alarmowy przełania → Discrete Input 0 ($L \geq 900$ l).
- **XV (Solenoid Valve)**: cyfrowy zawór odcinający spustu → Coil 0 (0 = zamknięty, 1 = otwarty).

2. Przygotowanie Środowiska Pracy

Krok 1: Weryfikacja i instalacja środowiska

Otwórz terminal (PowerShell / Bash) i sprawdź, czy sterownik protokołu jest już zainstalowany w systemie:

```
python -c "import pymodbus; print('[OK] Pymodbus zainstalowany')"
```

Jeżeli terminal zwróci błąd (ModuleNotFoundError), doinstaluj bibliotekę poleceniem:

```
pip install pymodbus
```

Krok 2: Uruchomienie emulacji obiektu (Plik server_tank.py)

Utwórz w edytorze plik server_tank.py, wklej poniższy kod symulatora i uruchom go w osobnym oknie terminala:

```
python server_tank.py
```

To okno emuluje fizykę zbiornika. Pozostaw je otwarte w tle przez cały czas trwania laboratorium!

```
1 import asyncio, struct
2 from pymodbus.server import StartAsyncTcpServer
3 from pymodbus.datastore import ModbusSequentialDataBlock,
   ModbusServerContext, ModbusSlaveContext
4
5 async def physics_loop(context):
6     slave = context[0]
```

```

7   level = 150.0 # Poziom początkowy [l]
8   while True:
9       await asyncio.sleep(0.1) # Krok dyskretyzacji dt = 100 ms
10      valve_open = slave.getValues(1, 0, count=1)[0] # Coil 0 (Zawor
        spustowy)
11
12      # Bilans masy: dopływ stały 1.8 l/s, odpływ 3.5 l/s po otwarciu
        zaworu
13      inflow, outflow = 0.18, (0.35 if valve_open else 0.0)
14      level = max(0.0, min(1000.0, level + inflow - outflow))
15
16      slave.setValues(4, 0, [int(level)]) # Input Reg 0:
        Poziom (LT)
17      slave.setValues(2, 0, [1 if level >= 900 else 0]) # Discrete In 0:
        Alarm (LSH)
18
19      # Holding Reg 1 i 2: Ciśnienie hydrostatyczne [bar] jako Float32 (
        IEEE 754)
20      p_bytes = struct.pack('>f', float(level / 200.0))
21      slave.setValues(3, 1, list(struct.unpack('>HH', p_bytes)))
22
23  async def main():
24      store = ModbusSlaveContext(
25          di=ModbusSequentialDataBlock(0, [0] * 10),
26          co=ModbusSequentialDataBlock(0, [0] * 10),
27          ir=ModbusSequentialDataBlock(0, [150] * 10),
28          hr=ModbusSequentialDataBlock(0, [500] * 10),
29      )
30      context = ModbusServerContext(slaves=store, single=True)
31      print("[SERVER] Obiekt fizyczny aktywny na porcie 5020...")
32      asyncio.create_task(physics_loop(context))
33      await StartAsyncTcpServer(context=context, address=("0.0.0.0", 5020))
34
35  if __name__ == "__main__":
36      asyncio.run(main())

```

Krok 3: Implementacja regulatora (Plik controller.py)

Utwórz drugi plik controller.py. W nim będziesz pisać kod klienta realizujący algorytmy sterowania z kolejnych modułów konspektu.

Typowe problemy i ich rozwiązania

- `OSError: [Errno 98/10048] Address already in use: Port 5020 jest już zajęty.` Oznacza to, że masz już uruchomiony proces serwera w tle. Zamknij poprzednie okno konsoli lub ubij proces Pythona.
- `ModbusIOException: [Input/Output] Connection refused: Klient nie może połączyć się z węzłem.` Upewnij się, że skrypt server_tank.py został uruchomiony jako pierwszy i nadal działa w osobnym oknie.
- `AttributeError: 'ModbusClient' object has no attribute ...:` Niezgodność wersji biblioteki. Wersja pymodbus 3.x używa nazw `read_input_registers`, `write_register` itp.

3. Część I: Hello Modbus (rozgrzewka)

Zadanie 1: Inspekcja węzła i zapis nastawy

Nawiąż połączenie z serwerem obiektu pod adresem 127.0.0.1:5020.

1. Uzupełnij kod, aby odczytać aktualny poziom cieczy z **Input Register 0**.
2. Odczytaj aktualną nastawę progu z **Holding Register 0**.
3. Nadpisz wartość w **Holding Register 0** nową wartością: **65**. Potwierdź ponownym odczytem.

Wskazówka programistyczna: obsługa odpowiedzi w pymodbus

Metody klienta w przypadku błędu (np. zły adres, brak łączności) nie rzucają tradycyjnego wyjątku w Pythonie, lecz zwracają obiekt błędu. Poprawność ramki można zweryfikować metodą:

```
if odp.isError():  
    # reakcja na błąd komunikacji
```

Dopiero gdy odpowiedź jest poprawna, bezpiecznie odczytujemy dane: `odp.registers[...]` dla rejestrów lub `odp.bits[...]` dla stanów binarnych.

```
1  import time  
2  from pymodbus.client import ModbusTcpClient  
3  
4  client = ModbusTcpClient('127.0.0.1', port=5020)  
5  if not client.connect():  
6      print("Błąd: brak połączenia z węzłem Modbus!")  
7      exit()  
8  
9  # TODO 1: Odczytaj 1 rejestr wejściowy pod adresem 0 (read_input_registers  
10 )  
11 # Wskazówka: res.registers[0] zawiera odczytana liczba całkowita  
12 # rr = client.read_input_registers(address=..., count=...)  
13  
14 # TODO 2: Odczytaj Holding Register pod adresem 0 (read_holding_registers)  
15  
16 # TODO 3: Zapisz nowa wartosc 65 do Holding Register 0 (write_register)  
17 client.close()
```

3.1 Cyberbezpieczeństwo OT: dlaczego infrastruktura krytyczna bywa bezbronna?

Modbus powstał pod koniec lat 70., gdy sieci przemysłowe stanowiły odizolowaną fizycznie magistralę szeregową (RS-485). Przeniesienie protokołu bezpośrednio na stos TCP/IP sprawiło, że **każdy, kto uzyska dostęp do segmentu sieciowego obiektu, może bez przeszkód sterować procesem fizycznym**. W architekturze protokołu nie przewidziano uwierzytelniania, szyfrowania ani kontroli integralności danych – sterownik PLC bezkrytycznie wykonuje każde polecenie zapisu jako autoryzowane polecenie operatora.

Przykład z realnego świata (Lwów, styczeń 2024 r.): podatność tę wykorzystano podczas ataku na przedsiębiorstwo ciepłownicze we Lwowie (Lvivteploenergo). Hakerzy uzyskali dostęp do sieci wewnętrznej i manipulowali punktami pomiarowo-sterującymi opartymi na protokole Modbus, co doprowadziło do paraliżu dostaw ciepła i ciepłej wody dla ponad 600 budynków mieszkalnych w środku zimy.

Źródło: [Veridify Security: Modbus Vulnerabilities Used for Cyberattack on a Heating Utility](#)

3.2 Czas Rzeczywisty w Automatyce: dlaczego nie zwykłe while True?

W informatyce panuje błędne przekonanie, że system czasu rzeczywistego (RT – *Real-Time*) to system działający „błyskawicznie”. W inżynierii i automatyce oznacza to coś innego: **gwarancję determinizmu czasowego**. Nieważne, czy krok wynosi 1 ms czy 500 ms – odpowiedź **musi** nastąpić w ściśle określonym przedziale czasowym T_s .

Dlaczego naiwne while True: ... time.sleep(0.5) jest błędem inżynierskim?

1. **Zjawisko dryftu zegara (Clock Drift):** funkcja sleep(0.5) usypia wątek na pół sekundy po wykonaniu kodu pętli. Jeśli zapytanie sieciowe przez Modbus i obliczenia trwają np. 40 ms, rzeczywisty okres pętli wyniesie:

$$T_{\text{rzeczywiste}} = T_{\text{obliczeń}} + T_{\text{sieci}} + T_{\text{sleep}} = 40 \text{ ms} + 500 \text{ ms} = 540 \text{ ms}$$

Po minucie pętla wykona się o kilka razy mniej, niż wynika z równań różnicowych regulatora!

2. **Rozwiązanie: kompensacja opóźnień:** zamiast stałego uśpienia, obliczamy bezwzględny punkt w czasie dla kolejnej próbki:

$$t_{\text{sleep}} = \max(0, t_{\text{następny}} - t_{\text{bieżący}})$$

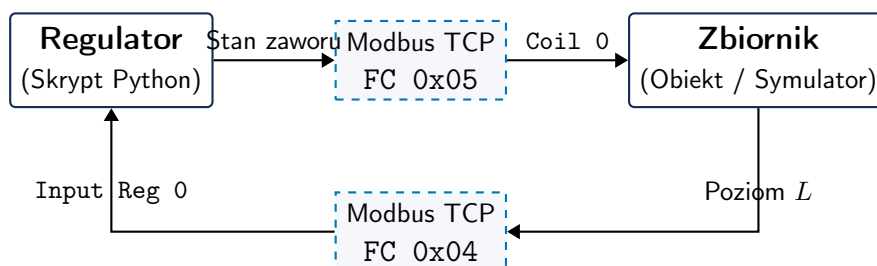
Dzięki temu jitter (zmiennosc opóźnień) zapytania sieciowego nie rozjeżdża okresu dyskretyzacji T_s .

3. **Zasada Fail-Safe (stan bezpieczny):** wypięcie pętli przez KeyboardInterrupt (Ctrl+C) w trakcie while True może zostawić otwarty zawór na zawsze. Blok finally to gwarancja wymuszenia stanu bezpiecznego obiektu przed zamknięciem procesu.

4. Część II: Układ Regulacji Poziomu w Zbiorniku

4.1 Struktura Pętli Regulacji Dwupołożeniowej

Układ pracuje w zamkniętej pętli sprzężenia zwrotnego. Komunikacja pomiędzy obiektem technologicznym (procesem) a algorytmem regulacji odbywa się wyłącznie za pośrednictwem ramek Modbus TCP:

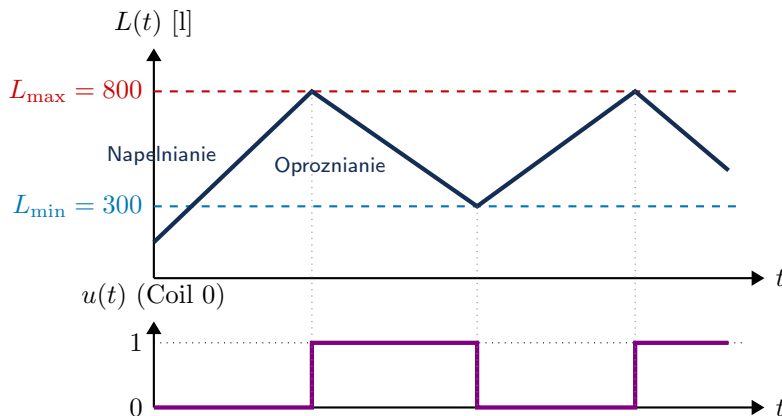


Charakterystyka statyczna elementu nieliniowego (histereza):

- $e(t) > L_{\max} \implies u(t) = 1$ (załączenie odpływu),

- $e(t) < L_{\min} \implies u(t) = 0$ (odcięcie odpływu),
- $L_{\min} \leq e(t) \leq L_{\max} \implies u(t) = u(t - T_s)$ (pamięć stanu poprzedniego).

Dynamika czasowa elementu nieliniowego (przebiegi $L(t)$ i $u(t)$):



4.2 Mapa Sygnałów Obiektowych (I/O Tag Table)

Wymiana danych pomiędzy pętlą regulacji a emulowanym obiektem technologicznym opiera się na następujących rejestrach magistrali:

Adres	Przestrzeń Modbus	Symbol P&ID	Typ	Opis inżynierski zmiennej
0	Input Register	LT-101	int16	Bieżący poziom cieczy $L \in [0, 1000]$ l
0	Discrete Input	LSH-101	bool	Sprzętowy alarm przełania (1 = przełanie $L \geq 900$ l)
0	Coil	XV-101	bool	Elektrozawór spustowy (0 = odcięty, 1 = otwarty)
1-2	Holding Register	PT-101	float32	Cięśnienie hydrostatyczne [bar] (dla zad. dodatkowego)

Zadanie 2: Implementacja regulatora dwupołożeniowego z histerezą

Sterowanie typu włącz/wyłącz bez histerezy niszczy elementy wykonawcze (tzw. migotanie styków/chattering). Twoim zadaniem jest napisanie algorytmu regulacji:

1. W nieskończonej pętli co 0.5 s odczytuj poziom cieczy ze zbiornika.
2. Otwórz zawór spustowy (Coil 0 $\rightarrow$ 1), dopiero gdy poziom przekroczy górną granicę $L_{\max} = 800$ l.
3. Zamknij zawór spustowy (Coil 0 $\rightarrow$ 0), dopiero gdy poziom opadnie poniżej $L_{\min} = 300$ l.

```

1 import time
2 from pymodbus.client import ModbusTcpClient
3
4 client = ModbusTcpClient('127.0.0.1', port=5020)
5 if not client.connect():
6     print("[ERROR] Brak komunikacji ze sterownikiem!")
7     exit(1)
8
9 SAMPLE_TIME = 0.5    # Krok dyskretyzacji Ts = 500 ms

```

```

10 valve_open = False # Pamięć stanu węzła
11 is_running = True
12
13 try:
14     next_tick = time.monotonic()
15
16     while is_running:
17         # TODO 1: Pobierz bieżący poziom cieczy (Input Register 0)
18         # Odpowiedz to kontener: dla rejestrów czytamy liste .registers,
19         # dla bitów .bits
20         # level = res.registers[0] | valve_state = res.bits[0]
21
22         # TODO 2: Zaimplementuj petle histerezy:
23         # - jeśli poziom > 800 i zawór zamknięty -> otwórz zawór (Coil 0)
24         # - jeśli poziom < 300 i zawór otwarty -> zamknij zawór (Coil 0)
25
26         # TODO 3: Wypisz bieżący stan w konsoli (np. print(f"Poziom: {
27         # level} l | Zawór: {valve_open}"))
28
29         # Kompensacja opoźnień magistrali (stały okres Ts):
30         next_tick += SAMPLE_TIME
31         time.sleep(max(0.0, next_tick - time.monotonic()))
32
33 except KeyboardInterrupt:
34     print("\n[STOP] Zatrzymano petle regulacji przez operatora.")
35
36 finally:
37     # TODO 4 (FAIL-SAFE): Wymus stan bezpieczny (otwarcie zaworu)
38     # i czyste zamknięcie gniazda TCP
39     # client.write_coil(address=0, value=True)
40     client.close()
41     print("[CLEANUP] Magistrala zamknięta bezpiecznie.")

```

5. Zadania dodatkowe i wyzwania inżynierskie

1. **Wizualizacja dynamiki w terminalu (Sparkline / Wykres):** zastąp wypisywanie pojedynczych liczb dynamicznym paskiem w konsoli, który rysuje poziom cieczy oraz progi załączenia:

```

1 bar = "#" * (level // 40)
2 print(f"[{level:4d} l] |{bar:<25}| [MIN: 300 | MAX: 800]")

```

2. **Dekodowanie Float32 (Standard IEEE 754):** pomiary temperatury czy ciśnienia w przemyśle przesyłane są zazwyczaj jako liczby zmiennoprzecinkowe w dwóch kolejnych 16-bitowych rejestrach trzymanyh. Odczytaj rejestry pod adresem 1 i 2, a następnie przekonwertuj je modulem struct:

```

1 import struct
2 # rr.registers zawiera np. [16968, 0]
3 raw_bytes = struct.pack('>HH', rr.registers[0], rr.registers[1])
4 floating_val = struct.unpack('>f', raw_bytes)[0]
5 print(f"Wartosc zmiennoprzecinkowa: {floating_val:.2f}")

```

Pod maską Pythona: Jak moduł struct rozmawia ze sprzętem?

Rejestry Modbus są ściśle 16-bitowe (`uint16`, zakres $0 \dots 65535$). Zmienne zmiennoprzecinkowe (np. ciśnienie w barach) w standardzie **IEEE 754 Single Precision** wymagają **32 bitów** (4 bajtów):



Serwer PLC rozcina liczbę 32-bitową na dwa 16-bitowe rejestry (Holding Reg 1 i 2). Moduł `struct` w Pythonie pozwala na bezpośrednią reinterpretację surowych bajtów w pamięci bez konwersji tekstowej:

- `struct.pack('>HH', reg0, reg1)` – pakuje dwa rejestry 16-bitowe (H = unsigned short) w 4 surowe bajty w porządku Big-Endian (>).
- `struct.unpack('>f', raw_bytes)[0]` – reinterpretuje te same 4 bajty jako liczbę zmiennoprzecinkową `float32` (f).

Znak > oznacza kolejność bajtów Big-Endian (najbardziej znaczący bajt na początku).

3. **Cyber-atak Man-in-the-Middle (Zabawa w segment sieci):** podłącz się pod adres IP sąsiada i uruchom równoległy skrypt, który w pętli wysyła `client.write_coil(0, False)`. Zaobserwuj zachowanie jego obiektu fizycznego: zbiornik zacznie przepełniać się mimo działającego algorytmu regulacji na maszynie kolegi.
4. **Destabilizacja układu czasem próbkowania (aliasing czasowy):** w automatyce algorytm może być w 100% poprawny matematycznie, a mimo to doprowadzić do awarii fizycznej obiektu ze względu na zbyt wolne próbkowanie.
 - Zwiększ krok próbkowania `SAMPLE_TIME` z 0.5 s kolejno do 2.0 s, 4.0 s oraz 6.0 s.
 - Zaobserwuj, co dzieje się z histerezą: czy regulator nadal utrzymuje poziom w przedziale $[300, 800]$ I?
 - Sprawdź odczyt sprzętowego czujnika przelania (**Discrete Input 0**). Dlaczego doszło do przekroczenia stanu krytycznego ($L \geq 900$ I), skoro w kodzie masz warunek otwarcia zaworu już przy 800 I?